Содержание

1 Сводка **2**

2 Введение **2**
2.1 (min,+)-свертка 2
2.2 Задача о рюкзаке 3
2.3 Структуры данных 4
2.3.1 Разреженная таблица 4
2.3.2 Модифицированная разреженная таблица 6
2.3.3 Дерево отрезков с модификацией 9

3 Основные результаты **11**
3.1 Задача о (min,+)-свертке с использованием разреженной таблицы 11
3.1.1 Субквадратичный алгоритм для разреженной таблицы 11
3.1.2 (min,+)-свертка для выпуклых и вогнутых функций 12
3.1.3 (min,+)-свертка для функции с k точками перегиба 13
3.1.4 (min,+)-свертка для полиномиальной функции 13
3.2 Задача о (min,+)-свертке с ипсользованием дерева отрезков с модификацией 13
3.2.1 (min,+)-свертка для выпуклых и вогнутых функций 14
3.2.2 (min,+)-свертка для функции с k точками перегиба 14
3.2.3 (min,+)-свертка для полиномиальной функции 14
3.3 Задача о рюкзаке 16
3.3.1 Техника динамического программироавния для задачи о рюкзаке . 16
3.3.2 Результаты для задачи о рюкзаке для функционала с ограничениями 17
3.3.3 Задача о рюкзаке для функционала в общем случае 18
3.3.4 Метод двойного экспоненциального представления 19
3.3.5 Задача о рюкзаке с методом двойного экспоненциального представления 20
3.3.6 Результаты эксперимента 28

4 Выводы **31**

5 Ссылки и использованная литература **32**

1 Сводка

В работе рассматривается проблема вычисления (min,+)-свертки двух последовательностей a и b длины n и m соответственно, дополнительно полагаем, что $n \geqslant m$. Полагаем, что последовательность a задана произвольно, а последовательность $b : b_i = f(i)$, где $f(x) : [0, m) \to \mathbb{R}$ - функция, обладающая одним из следующих свойств:

1. $f(x)$ - выпуклая функция, т.е. $f(i+1) - f(i) \geqslant f(i) - f(i-1)$, для любых i;
2. $f(x)$ - вогнутая функция, т.е. $f(i+1) - f(i) \leqslant f(i) - f(i-1)$, для любых i;
3. $f(x)$ - функция с k точками перегиба;
4. $f(x)$ - полиномиальная функция, т.е. $f \in \mathbb{Z}^d[x]$, для некоторого фиксированного значения d.

Для данных случаев были получены субквадратичные алгоритмы. Были улучшены результаты для техники, представленной в [1]. Была рассмотрена новая техника. Полученные результаты применяются в задаче о рюкзаке, где целевой функционал - сепарабельная функция. Также был программно реализован алгоритм для задачи о рюкзаке в общем случае и проведены вычислительные эксперименты для описанного в [3] алгоритма с модификацией.

Цели работы

1. Исследование задачи о (min,+)-свертке двух последовательностей при ограничениях на одну из последовательностей;
2. Применение полученных результатов для построения субквадратичного алгоритма в задаче о рюкзаке;
3. Проведение экспериментов с применением метода двойного экспоненциального представления коэффициентов для задачи о рюкзаке в общем случае.

2 Введение

2.1 (min,+)-свертка

Определение 1 (min,+)-сверткой последовательностей $a = \{a_0, ..., a_n\}$ и $b = \{b_0, ..., b_m\}$ называем последовательность $c = (a \star b)$, определяемую следующей формулой:

$$c_k = \min_{i+j=k} \{a_i + b_j\}, \text{ для } k \in \{0, ..., n+m-2\}.$$

В данной работе было удобнее рассматривать другую задачу, которая может быть за линейное время сведена к задаче о (min,+)-свертке.
Пусть заданы две последовательности $a = \{a_0, ..., a_n\}$ и $b = \{b_0, ..., b_m\}$. Необходимо вычислить последовательность $c := a \bullet b$, определяемую следующей формулой:

$$c_k = \min_{0 \leqslant i \leqslant m-1} \{a_{k+i} + b_i\}, \text{ для } k \in \{0, ..., n-m\}$$

Далее в работе под задачей о (min,+)-свертке будем понимать второй вариант постановки. Для исключения неоднозначности будет оставлено уточнение или комментарий, если используется классическое определение или иная постановка задачи.
Для задачи о (min,+)-свертке неизвестно, существует ли субквадратичный алгоритм в

общем случае. Известен тривиальный алгоритм, вычисляющий коэффициенты (min,+)-свертки за $O(n^2)$. Данный результат может быть улучшен до $O(n^2/2^{\Omega(\sqrt{\log n})})$ сведением задачи к вычислению (min,+)-матричному умножению, которое описано в [2].
Результат, полученный в [2] может быть улучшен в некоторых специальных случаях.
Пусть $b_i = f(i)$, для некоторой функции $f(x) : [0, m) \to \mathbb{R}$. Будем рассматривать случаи, в которых $f(x)$ обладает одним из следующих свойств:

1. $f(x)$ - выпуклая функция, т.е. $f(i+1) - f(i) \geqslant f(i) - f(i-1)$, для любых i;
2. $f(x)$ - вогнутая функция, т.е. $f(i+1) - f(i) \leqslant f(i) - f(i-1)$, для любых i;
3. $f(x)$ - функция с k точками перегиба;
4. $f(x)$ - полиномиальная функция, т.е. $f \in \mathbb{Z}^d[x]$, для некоторого фиксированного значения d.

Определение 2 Будем полагать, что значение функции $f(x)$ может быть получено обращением к оракулу EV. Для заданных $f(x)$ и $x \in \mathcal{D}(f)$ этот оракул возвращает значение $f(x)$. Также дополнительно полагаем, что для двух заданных $x, y \in \mathcal{D}(f)$ этот оракул может сравнить значения $f(x)$ и $f(y)$.
Оракул EV используется при вычислении (min,+)-свертки для каждого из представленных выше случаев. Поэтому для удобства под терминами "сложность"и "вычислительная сложность"будем подразумевать арифметическую и оракульную сложности, т.е. количество обращений к оракулу и количество элементарных арифметических операций.
В следующей таблице представлены полученные результаты для указанных выше случаев, а также сравнение с предыдущими результатами, полученными в [1]:

Случай	**Сложность**	**Предыдущие результаты**
f вогнутая/выпуклая	$O(n^{\frac{5}{3}} \log n)$	$O(n^{\frac{4}{3}} \log^2 n)$
f имеет k точек перегиба	$O(k \cdot n^{\frac{4}{3}} \log^2 n)$	Новый результат
$f \in \mathbb{Z}^d[x]$	$O(d \cdot n^{\frac{4}{3}} \log^2 n)$	$O(d^3 \cdot n^{1+\log_2(d)+\frac{1}{1+\log_2(d)}} \log^2(n))$

Подробное описание и доказательство полученных результатов представлено в разделе 3.

2.2 Задача о рюкзаке

Пусть $\omega \in \mathbb{Z}^n_{>0}$, $u \in \mathbb{Z}^n_{>0}$ и $f : \mathbb{Z} \to \mathbb{R}$ - сепарабельная функция. $f(x) = f_1(x_1) + ... + f_n(x_n)$, где $f_i(x) : \mathbb{Z} \to \mathbb{R}$.

Определение 3 Под задачей о рюкзаке будем понимать следующую задачу:

$$f(x) \to max$$

$$\begin{cases} \omega^\top x = W \\ 0 \leqslant x \leqslant u \\ x \in \mathbb{Z}^n. \end{cases}$$

В работе рассматриваются специальные случаи, где $f(x)$ задана следующим образом:

1. $f(x)$ - выпуклая функция, т.е. $f(i+1) - f(i) \geqslant f(i) - f(i-1)$, для любых i;
2. $f(x)$ - вогнутая функция, т.е. $f(i+1) - f(i) \leqslant f(i) - f(i-1)$, для любых i;
3. $f(x)$ - функция с k точками перегиба;
4. $f(x)$ - полиномиальная функция, т.е. $f \in \mathbb{Z}^d[x]$, для некоторого фиксированного значения d.

Для случая, когда $f(x)$ выпуклая/вогнутая известен следующий результат: $O(n \cdot W^2)$ арифметические операции, основанный на динамическом программировании. Случаи, когда $f(x)$ полиномиальная были рассмотрены в [1] и улучшены в этой работе. Случай функции с k точками перегиба в [1] рассмотрен не был.
В следующей таблице представим полученные результаты:

Случай	**Сложность**	**Предыдущие результаты**
f вогнутая/выпуклая	$O(n \cdot W^{\frac{5}{3}} \log n)$	$O(n \cdot W^{\frac{4}{3}} \log^2 n)$
f имеет k точек перегиба	$O(k \cdot n \cdot W^{\frac{4}{3}} \log^2 n)$	Новый результат
$f \in \mathbb{Z}^d[x]$	$O(d \cdot n \cdot W^{\frac{4}{3}} \log^2 n)$	$O(d^3 \cdot n \cdot W^{\sigma} \log^2(W))$, где $\sigma = 1 + \log_2(d) + \frac{1}{1+\log_2(d)}$

Подробное описание и доказательство полученных результатов представлено в разделе 3.

2.3 Структуры данных

В этом разделе опишем используемые структуры данных, которые используются для алгоритмов. В [1] были использованы оцередь с поддержкой минимума для случая $f(x)$ - линейная функция и модификация дерева орезков для случаев, когда $f(x)$ - выпуклая/вогнутая или полиномиальная функция. Подробное описание очереди с поддержкой минимума, дерева отрезков и дерева отрезков с модификацией можно найти в [1]. Дерево отрезков с модификацией будет описано ниже, так как оно используется для получения итогового результата для вычисления (min,+)-свертки и задачи о рюкзаке, в случае, когда одна из последовательностей - полиномиальная функция.
Ввиду статичности входных данных операция обновления в дереве отрезков не используется, поэтому в данной работе применяется другая структура данных, использующаяся для получения минимума на отрезке - разреженная таблица (Sparse Table). В работе строится модификация данной структуры данных для вычисления (min,+)-свертки.

2.3.1 Разреженная таблица

Разреженная таблица - структура данных, позволяющая на отрезке получать результат идемпотентной операции за константную сложность и требующая $O(n \log n)$ затрат по памяти и количеству операций для построения структуры. В работе рассматриваем разреженную таблицу для получения минимума на отрезке.

Определение 4 Будем называть интервал $[i, j)$ базовым, если $[i, j) = [i \cdot 2^{d-k}, (i+1) \cdot 2^{d-k}$ для некоторых $i \in \{0, ..., 2^{k-1}\}$ и $k \in \{0, ..., d\}$.

Описание структуры

Разреженная таблица - структура данных, предназначенная для получения минимума на отрезке при статичных данных. В работе приводится описание построения данной структуры. Идея данной структуры данных заключается в хранении ответов для отрезков. В случае хранения ответов для всевозможных отрезков нам потребовалось бы $O(n^2)$ памяти. Разреженная таблица хранит ответы не для всех отрезков, а только для всевозможных отрезков длины 2^k, где $2^k \leqslant d$. Отрезок таблицы $T[i][j]$ хранит значение для отрезка $[i, i + 2^j - 1]$.
Рассмотрим экземпляр разреженной таблицы T для массива A длины n. Для удобства без ограничения общности, будем полагать, что $n = 2^d$ для некоторого $d \in \mathbb{N}$. Приведем список операций, которые может выполнять T и асимптотическую сложность этих операций:

1. операция $T.build(A)$ - построение разреженной таблицы для массива A длины n. Сложность в худшем случае $O(n \log n)$;

2. операция $T.\min(i, j)$ - получение минимума на подмассиве $A[i, j)$. Сложность в худшем случае $O(1)$.

Построение разреженной таблицы
Идея построения заключается в том, что любой отрезок, чья длина больше 1 может быть разбит на 2 отрезка меньшей длины. А значение минимума на отрезке длины 1 это и есть само значение элемента.
Опишем построение минимума на отрезке $[i, i+2^j-1]$. Он равен минимуму из значений на отрезках $[i, i+2^{j-1}-1]$ и $[i+2^{j-1}, i+2^j-1]$. Отсюда получаем выражение для вычисления элементов таблицы:

$$T[i][j] = \min(T[i][j-1], T[i+2^{j-1}][j-1]) \tag{1}$$

Приведем программную реализацию операции построения разреженной таблицы:

```
vector<vector<int>> build_sparse_table(const vector<int>& arr) {
    int n = arr.size();
    int maxpow = log2(n + 1);
    vector<vector<int>> T(vector<int>(n, 0), maxpow);
    sparse[0] = arr;
    for (int k = 0; k + 1 < maxpow; k++) {
        for (int i = 0; i + (1 << k) < n; i++) {
            T[ind][k + 1] = min(T[ind][k], T[ind + (1 << k)][k]);
        }
    }
    return T;
}
```

Далее необходимо построить массив, который для каждой длины отрезка будет определять отрезками какой длины его стоит представлять:

```
int lg[n + 1];
lg[1] = 0;
for (int i = 2; i <= n; i++)
    lg[i] = lg[i / 2] + 1;
```

Далее приведем программную реализацию получения ответа на запрос:

```
int get_answer(int l, int r)
{
    int power = lg[r - l];
    return min(T[l][power], T[r - (1 << power)][power]);
}
```

Для построения T нам потребовалось $O(n \log n)$ памяти и операций. По построению функции, возвращающей ответ на запрос легко убедиться, что ее сложность составляет $O(1)$.

2.3.2 Модифицированная разреженная таблица

При построении модификации разреженной таблицы для массива A длины n будем также полагать, что $n = 2^d$ для некоторого $d \in \mathbb{N}$.
Пусть задан массив A длины n и функция $f(x) : [0, n) \to \mathbb{R}$. Задан $x \in \{0, ..., n-1\}$, а также $i, j \in \{0, ..., n\}$. Необходимо вычислить значение следующей функции:

$$\mathcal{F}([i, j); x) = \min\{A[i] + f(x), A[i+1] + f(x+1), ..., A[i + (j-i-1)] + f(x + (j-i-1))\},$$

$$\text{т.е. } \mathcal{F}([i, j); x) = \min_{0 \leqslant k \leqslant j-i} \{A[i+k] + f(x+k)\}$$

Полагаем, что для вычисления значений $f(x)$, где $x \in [0, n)$ мы можем использовать EV.

Определение 5 Пусть $\mathcal{I} = [\alpha, \beta)$ - некоторый интервал, $\alpha, \beta \in \mathbb{R}$, $\{\mathcal{I}_i = [\alpha, \beta)\}$ - множество интервалов, $\alpha_i, \beta_i \in \mathbb{R}$, $i \in \{0, ..., k\}$ обладающих следующим свойством:

$$\mathcal{I} = \cup \mathcal{I}_i.$$

Будем говорить, что $\mathcal{I}_i$ образуют разбиение интервала $\mathcal{I}$.
По построению функции $\mathcal{F}([i, j); x)$ и из идемпотентности операции взятия минимума следует утверждение:

Утверждение 1 Пусть интервал $[i, j)$ может быть разбит на интервалы $[i_1, j_1)$ и $[i_2, j_2)$: $[i, j) = [i_1, j_1) \cup [i_2, j_2)$, где $i_2 >= i_1$. Тогда

$$\mathcal{F}([i, j); x) = \min\{\mathcal{F}([i_1, j_1); x), \mathcal{F}([i_2, j_2); x + j_1 - i_1)\}.$$

Определение 6 Интервал $[a, b) \subseteq \mathbb{R}$ будем называть целочисленным, если его концы $a, b \in \mathbb{Z}$.

Определение 7 Пусть $f : [0, n) \to \mathbb{R}$ и $\mathcal{I} \subseteq [0, n)$ - целочисленный интервал. Пусть $\mathcal{I}_1, ..., \mathcal{I}_m$ -множество целочисленных интервалов, обладающих следующими свойствами:

1. интервалы $\mathcal{I}_1, ..., \mathcal{I}_m$ образуют разбиение $\mathcal{I}$;
2. для любых j и $x \in \mathcal{I}_j$ функция $f(x)$ удовлетворяет одному и только одному из следующих условий: $f(x) > 0, f(x) < 0, f(x) \geqslant 0, f(x) \leqslant 0$.

Минимальное множество таких интервалов $\mathcal{I}_1, ..., \mathcal{I}_m$ будем называть минимальным знаковым разбиением для функции f. Множество всех минимальных знаковых разбиений будем обозначать $\mathcal{P}(f, \mathcal{I})$.

Определение 8 Пусть задана функция f и некоторый интервал $\mathcal{I} \subseteq \mathcal{D}(f)$. Будем считать, что задан оракул SP, который для заданной f и $\mathcal{I}$ возвращает некоторое минимальное знаковое разбиение из множества $\mathcal{P}(f, \mathcal{I})$.

Определение 9 Пусть $f : \mathcal{I} \to \mathbb{R}$, где $\mathcal{I}$ - целочисленный интервал. Рассмотрим функцию $g_{ab}(x) = f(x+a) - f(x) + b$, где $a \in \mathbb{Z}_{\geqslant 0}$ и $b \in \mathbb{Z}$.
Будем называть величину $p_f(a, b, \mathcal{J})$ максимальным знаковым разбением функции g_{ab} на интервале $\mathcal{J}$ для всевозможных значений a и b и всевозможных целочисленных подинтервалов $\mathcal{J} \subseteq \mathcal{I}$.

$$p_f(a, b, \mathcal{J}) = \max\{|P| : P \in \mathcal{P}(g_{ab,\mathcal{J}})\}$$

Теорема 1 Пусть задана функция $f : [0, n) \to \mathbb{R}$, задан массив A длины n, причем $n = 2^d$ для некоторого $d \in \mathbb{N}$, также полагаем, что задан максимальный размер знакового разбиения для функции $f : p := p_f$. Полагаем, что EV и SP поддерживаются. Тогда можем посторить структуру данных T, являющуюся разреженной таблицей с модификацией, которую будем называть также разреженной таблицей, которая поддерживает следующие операции:

1. операция $T.build(A)$ создает экземпляр разреженной таблицы для массива A длины n. Вычислительная сложность и сложность по памяти в худшем случае

$$O(n^{2+\log_2(p)} \cdot \log_2(n))$$

2. операция $T.query(i, j, x)$ возвращает значение функции $\mathcal{F}([i, j), x)$. Сложность в худшем случае $O(\log n)$.

Вызовы SP доступны для функций вида $g(x) = f(x+a) - f(x+b) + c$, где $a, b \in \mathbb{Z}_{\geqslant 0}$ и $c \in \mathbb{Z}$. Вызовы EV доступны для f. Далее под разреженной таблицей понимаем именно разреженную таблицу с модификацией. Если описывается классический вариант разреженной таблицы, это будет указано.

Доказательство

Описание структуры данных
Новая структура данных является разреженной таблицей с дополнительной информацией, которую будет хранить каждый элемент.
Дополним каждый элемент v конечным множеством $\mathcal{G}_v$ функций $g : \mathcal{D}_g \cap \mathbb{Z} \to \mathbb{Z}$, где каждая область определения $\mathcal{D}_g$ - целочисленный подинтервал интервала $[0, n)$ и g на своей области определения дейслвует следующим образом: $g(x) = A[j] + f(x+t)$ для некоторых $j, t \in \{0, ..., n-1\}$.
Для каждого элемента таблицы $v \in T$ будем поддерживать следующие инварианты:

1. Инвариант 1: интервалы $\mathcal{D}_g : \forall g \in \mathcal{G}_v$ являются разбиением $[0, n)$

$$[0, n) = \bigsqcup_{g \in \mathcal{G}_v} \mathcal{D}_g;$$

2. Инвариант 2: $\forall x \in [0, n)$ существует единственная функция $g \in \mathcal{G}_v$, такая что $x \in \mathcal{D}_g$ и

$$\mathcal{F}([i_v, j_v), x) = g(x);$$

3. Инвариант 3: функции $g \in \mathcal{G}_v$ отсортированы в неубывающем порядке по отношению к границам их областей определения $\mathcal{D}_g$;

4. Инвариант 4: $\forall g \in \mathcal{G}_v$ функция $g(x)$ действует на $\mathcal{D}_g$ следующим образом: $g(x) = A[j] + f(x+t)$ для $j, t \in \{0, ..., n-1\}$;

Описание запроса для интервала, который хранится в таблице
Пусть поступил запрос $T.query(i_v, j_v, x)$ для отрезка, который является элементом таблицы v. По второму инварианту нам требуется найти подходящую функцию $g \in \mathcal{G}_v$. По третьему инварианту все функции в $\mathcal{G}_v$ отсортированы в неубывающем порядке по отношению к границам их областей определения, поэтому подходящая функция g может быть найдена в худшем случае за $O(\log n)$ операций, т.к. $\mathcal{G}_v$ содержит не более n функций. По 4 инварианту $g(x) = A[j] + f(x+t)$, поэтому она может быть вычислена вызовом EV.

Описание запроса для интервала в общем случае
Пусть поступил запрос $T.query(i, j, x)$ такой, что интервал $[i, j)$ не хранится в таблице. Тогда по преподсчитанным длинам отрезков и (1), а также зная границы запроса i и j можем выбрать два элемента T, кодирующие интервалы $[i, j_1)$ и $[i_2, j)$ такие, что $[i, j) = [i, j_1) \cup [i_2, j)$. По предыдущему пункту получаем ответы для каждого из базовых интервалов. Ответом для исходного интервала будет минимум из ответов для полученных базовых интервалов. Итоговая сложность $O(\log n)$.

Описание построения разреженной таблицы
Для начала строим классический вариант разреженной таблицы без модификации, описанной в 2.3.1 для массива A. Это займет $O(n \log n)$ затрат по памяти и по количеству операций.
Далее необходимо вычислить для каждого элемента v полученной таблицы T множество $\mathcal{G}_v$ и при этом необходимо соблюдать все указанные инварианты из описания структуры данных. Для этого построим и опишем рекурсивный алгоритм. Будем спускаться по таблице вниз и для каждого элемента, хранящего интервал длины 2^k будем подсчитывать ответ через элементы, хранящие интервалы длины 2^{k-1}. Пусть v описывает интервал, длина которого равна 1. Тогда для v $\mathcal{F}([i_v, j_v), x) = A[i_v] + f(x)$. Следовательно $\mathcal{G}_v$ содержит единственную функцию $g(x) = A[i_v] + f(x)$ и $\mathcal{D}_g = [0, n)$.
Теперь полагаем, что элемент v хранит информацию об интервале $[i_v, j_v)$, длина которого больше 1. По построению T и из изначального предположения о том, что длина массива A является степенью числа 2 очевидно, что $[i_v, j_v)$. Тогда существуют два элемента в таблице u и w, такие, что $[i_u, j_u) \cup [i_w, j_w) = [i_v, j_v)$. Для u и w уже ранее были вычислены $\mathcal{G}_u$ и $\mathcal{G}_w$.
Пусть P_u и P_w - множества граничных точек областей определения функций из $\mathcal{G}_u$ и $\mathcal{G}_w$ соответственно. $|P_u| = |\mathcal{G}_u| + 1$, $|P_w| = |\mathcal{G}_w| + 1$, т.к. $0, n \in P_u$ и $0, n \in P_w$. По 3 инварианту P_u и P_w отсортированы. P_v мы можем получить, объединив P_u и P_v. Это потребует $O(n)$ операций в худшем случае, при этом все 4 инварианта для P_v поддерживаются.
Рассмотрим пару соседних граничных точек $\nu, \tau \in P_v$. По второму инварианту существуют и единственны функции $g_u \in \mathcal{G}_u$ и $g_w \in \mathcal{G}_w$ такие, что $[\nu, \tau) \subseteq \mathcal{D}_{g_u} \cap \mathcal{D}_{g_w}$. По первому утверждению для $x \in [\nu, \tau)$

$$\mathcal{F}([i_v, j_v), x) = min\{g_u(x), g_w(x + j_u - i_u)\}.$$

Пусть $h(x) = g_u(x) - g_w(x + j_u - i_u)$, определенная на $[\nu, \tau)$. По инварианту 4 функция $h(x)$ имеет вид $f(x+a) - f(x+b) + c$ для некоторых $a, b \in \mathbb{Z}_{\geqslant 0}$ и $c \in \mathbb{Z}$.
Для преподсчета $\mathcal{F}([i_v, j_v), x)$ для $x \in [\nu, \tau)$ необходимо вычислить минимальное знаковое разбиение $\mathcal{S} \in \mathcal{P}(h, [\nu, \tau))$. Для этого мы вызываем SP.
Таким образом для каждого интервала $\mathcal{I} \in \mathcal{S}$: если $h(x) \geqslant 0$ на $\mathcal{I}$, то $\mathcal{F}([i_v, j_v), x) = g_u(x)$, если $h(x) \leqslant 0$, то в качестве $\mathcal{F}([i_v, j_v), x)$ берем функцию $g_w(x + j_u - i_u)$. Последовательно для всех интервалов $\mathcal{I}$ вычисляем функцию $g_{\mathcal{I}}$ и добавляем ее в $\mathcal{G}_v$ в указанном в 3

инварианте порядке. По условию теоремы наибольший размер минимального знакового разбиения p, поэтому каждый подинтервал $[\nu, \tau)$ разбивается не более чем на p интервалов, также такое же число функций будет добавлено в $\mathcal{G}_v$.
Далее оценим вычислительную сложность и затраты по памяти, используемые при построении. Для каждой пары последовательных точек $[\nu, \tau)$ из P_v добавляется не более p функций в $\mathcal{G}_v$. Следовательно

$$|\mathcal{G}_v| \leqslant (|\mathcal{G}_u| + |\mathcal{G}_w| - 1) \cdot p.$$

Обозначим $N(m) = max\{|\mathcal{G}_v| : j_v - i_v = m\}$, где m является степенью двойки. Так как $N(1) = 1$ имеем:

$$N(m) \leqslant 2 \cdot N(m/2) \cdot p \leqslant (2p)^{\log_2 m} = m^{1+\log_2 p}.$$

Также $N(m) \leqslant n$. Отсюда получаем

$$N(m) \leqslant \min\{m^{1+\log_2 p}, n\}.$$

Остюда заключаем, что вычислительная сложность построения $\mathcal{G}_v$ для элемента v составляет $O(\min\{m^{1+\log_2 p}, n\})$.
Далее оценим сложность построения всей разреженной таблицы T и затраты по памяти для ее хранения. Обозначим $t_v(m)$ затраты по памяти для хранения одного элемента таблицы, а $t(n)$ затраты по памяти для хранения всей таблицы.
Затраты по памяти для хранения всей таблицы можно оценить следующей формулой:

$$t(n) = \sum_{k=0}^{\log_2 n} 2^k \cdot t_v(n - 2^k).$$

Вычислим оценку асимптотической сложности:

$$t(n) < n^{1+\log_2(p)} \cdot \sum_{k=0}^{\log_2 n} 2^k < n^{2+\log_2(p)} \cdot \log_2(n)$$

Итоговая верхняя оценка вычислительной сложности и сложности по памяти составила

$$O(n^{2+\log_2(p)} \cdot \log_2(n)).$$

Теорема доказана. □

2.3.3 Дерево отрезков с модификацией

Дерево отрезков это структура данных, используемая для выполнения ассоциативных операций на отрезке, с поддержкой операции обновления в худшем случае за логарифмическую сложность.
Пусть имеется экземпляр дерева отрезков S. Для S определены следующие операции:

1. операция $S.build(A)$ создает дерево отрезков для массива A длины n. Сложность в худшем случае $O(n)$;

2. операция $S.query(i, j)$ возвращает ответ на запрос для подинтервала $[i, j) \in [0, n)$. Сложность в худшем случае $O(\log(n))$;

3. операция $S.update(i, x)$ изменяет значение элемента $A[i]$ на значение x. Сложность в худшем случае $O(\log(n))$.

Для вычисления (min,+)-свертки модифицируем дерево отрезков. Будем полагать, что массив A имеет длину $n = 2^d$ для некоторого $d \in \mathbb{Z}$. Полагаем, что задана функция $f(x) : [0, n) \to \mathbb{R}$. Заданы $x \in \{0, ..., n-1\}$ и $i, j \in \{0, ..., n\}$. Необходимо вычислить значение следующей функции:

$$\mathcal{F}([i, j); x) = \min\{A[i] + f(x), A[i+1] + f(x+1), ..., A[i + (j-i-1)] + f(x + (j-i-1))\},$$

$$\text{т.е. } \mathcal{F}([i, j); x) = \min_{0 \leqslant k \leqslant j-i} \{A[i+k] + f(x+k)\}$$

Полагаем, что для вычисления значений $f(x)$, где $x \in [0, n)$ мы можем использовать EV.

Теорема 2 Пусть задана функция $f : [0, n) \to \mathbb{R}$, задан массив A длины n, причем $n = 2^d$ для некоторого $d \in \mathbb{N}$, также полагаем, что задан максимальный размер знакового разбиения для функции $f : p := p_f$. Полагаем, что EV и SP поддерживаются. Тогда можем посторить структуру данных S, являющуюся деревом отрезков с модификацией, которая обозначена в [1] как *augmented segment tree*, которая поддерживает следующие операции:

1. операция $S.build(A)$ создает экземпляр дополненного дерева отрезков для массива A длины n. Вычислительная сложность и сложность по памяти в худшем случае

$$O(n^{\log_2(p) + \frac{1}{1+\log_2(p)}})$$

2. операция $S.query(i, j, x)$ возвращает значение функции $\mathcal{F}([i, j), x)$. Сложность в худшем случае $O(\log^2(n))$.

Вызовы SP доступны для функций вида $g(x) = f(x+a) - f(x+b) + c$, где $a, b \in \mathbb{Z}_{\geqslant 0}$ и $c \in \mathbb{Z}$. Вызовы EV доступны для f.

Доказательство

Доказательство теоремы можно найти в [1] в разделе 2.3 Теорема 1. □

3 Основные результаты

3.1 Задача о (min,+)-свертке с использованием разреженной таблицы

Далее приведем описание техники вычисления (min,+)-свертки при помощи разреженной таблицы, представим и докажем полученые результаты.
Рассмотрим задачу о (min,+)-свертке для следующих случаев:

1. f - выпуклая/вогнутая;
2. f имеет k точек перегиба;
3. $f \in \mathbb{Z}^d[x]$.

3.1.1 Субквадратичный алгоритм для разреженной таблицы

Рассмотрим функцию $f : [0, m) \to \mathbb{R}$. Полагаем, что она поддерживает вызовы EV и SP. Вычислим арифметическую и оракульную сложность для вычисления (min,+)-свертки. Для этого воспользуемся разреженной таблицей.
Так как полученные в Теореме 1 оценки для хранения и вычисления разреженной таблицы очень велики мы будем разбивать массив A на n/B последовательных блоков размера B. Без ограничения общности полагаем, что n и B являются степенями числа 2.
Для каждого блока B_j, $j \in \{1, ..., n/B\}$ будем строить разреженную таблицу T_j вызовом $T_j.build(A[B_j])$. По Теореме 1 нам потребуется

$$O(\frac{n}{B} \cdot B^{2+\log_2(p)} \cdot \log_2(B)) = O(n \cdot B^{1+log_2(p)} \cdot \log_2(B)) \tag{2}$$

затрат по времени и памяти.
Алгоритм включает $n - m$ шагов. На k-ом шаге мы будем вычислять c_k используя

$$c_k = \mathcal{F}([k, k + m), 0).$$

Пусть $B_j, ..., B_{j+t}$ будут последовательными блоками, затронутыми интервалом $[k, k+m)$, где $t = O(m/B)$. Если $t = 1$, то мы можем вычислить

$$c_k = T_j.query(k - B \cdot (j - 1), k - B \cdot (j - 1) + m, 0).$$

Теперь рассматриваем случай, когда $t \geqslant 2$. Пусть $s = j \cdot B - k$ размер пересечения B_j с интервалом $[k, k + m)$.
Тогда
$c_k := \mathcal{F}([k, k + m), 0) =$
$= \min \{\mathcal{F}(B_j \cap [k, k + m), 0), \mathcal{F}(B_{j+1} \cap [k, k + m), s), ..., \mathcal{F}(B_{j+t} \cap [k, k + m), s + B \cdot (t - 1))\}$.
Интервал $B_j \cap [k, k + m)$ - суффикс B_j. Поэтому

$$\mathcal{F}(B_j \cap [k, k + m), 0)$$

может быть вычислено вызовом $T_j.query(B - s, B, 0)$.
Интервал $\mathcal{F}(B_{j+t} \cap [k, k + m)$ - префикс B_{j+t}. Поэтому

$$\mathcal{F}(B_{j+t} \cap [k, k + m), s + B \cdot (t - 1))$$

может быть вычислено вызовом $T_{j+t}.query(0, k + m - B \cdot (t - 1), s + B \cdot (t - 1))$.
Средние значения $\mathcal{F}(B_{j+i}, s + B \cdot (i - 1))$ могут быть вычислены вызовом

$$T_{j+i}.query(0, B, s + B \cdot (i-1)).$$

Таким образом, c_k может быть вычислено как минимум среди всех полученных значений за

$$O(\frac{m}{B} \cdot \log_2(n)). \tag{3}$$

Из (2) и (3) сложность $n - m$ шагов с учетом построения составляет

$$O(n \cdot B^{1+log_2(p)} \cdot \log_2(B) + \tfrac{n^2}{B} \cdot \log_2(n)).$$

Для удобства обозначим $\sigma - 1 = 1 + log_2(p)$
Сбалансируем полученную формулу через следующее равенство:

$$n \cdot B^{\sigma-1} = \tfrac{n^2}{B}.$$

Отсюда получаем

$$B^{\sigma} = n.$$

Отсюда для балансировки размер B следует положить равным $n^{\frac{1}{\sigma}} = n^{\frac{1}{2+\log_2(p)}}$.
Итоговая сложность

$$O(n^{2-\frac{1}{2+\log_2(p)}} \cdot \log_2(n)) \tag{4}$$

Затраты по памяти

$$O(n^{2-\frac{1}{2+\log_2(p)}} \cdot \log_2(n^{\frac{1}{2+\log_2(p)}})).$$

3.1.2 (min,+)-свертка для выпуклых и вогнутых функций

Лемма 1 Пусть функция $f : [\alpha, \beta) \to \mathbb{R}$ является выпуклой или вогнутой, где $\alpha, \beta \in \mathbb{R}$. Также полагаем что f поддерживает EV. Тогда из [1] $p_f \leqslant 2$ и минимальное знаковое разбиение $\mathcal{P}(g, \mathcal{I})$ для заданного интервала $\mathcal{I}$ может быть вычислено за $O(\log n)$ вызовов EV.
Доказательство можно найти в [1] в секции 3.3 Лемма 3.

Теорема 3 Пусть $f : [0, m) \to \mathbb{R}$ - выпуклая/вогнутая функция. Будем полагать, что f поддерживает вызовы EV.
Тогда (min,+)-свертка может быть вычислена с арифметической и оракульной сложностью

$$O(n^{5/3} \cdot \log_2(n)).$$

Доказательство

Расмотрим функцию $g(x) = f(x + a) + b - f(x)$ и $\mathcal{I} \subseteq [0, n)$. По Лемме 1 вычисление минимального знакового разбиения для пары $(g, \mathcal{I})$ ограничены $O(\log n)$ вызовами EV, а $p := p_f \leqslant 2$. Используя оценку для вызова SP и формулу (4) получаем, что итоговая сложность составляет

$$O(n^{5/3} \cdot \log_2(n))$$

Теорема доказана. □

3.1.3 (min,+)-свертка для функции с k точками перегиба

Теорема 4 Пусть $f : [0, n) \to \mathbb{R}$ функция, имеющая k точек перегиба. Будем полагать, что f поддерживает вызовы EV и имеет непрерывные производные до 2 порядка включительно.
Тогда (min,+)-свертка последовательностей $a := A$ и $b := b_i = f(i)$ может быть вычислена с арифметической и оракульной сложностью

$$O(k \cdot n^{5/3} \cdot \log_2(n)).$$

Доказательство

Так как функция f имеет k точек перегиба следовательно на $[0, n)$ имеем $k + 1$ участок $\mathcal{I}_i : f(x)$ выпукла или вогнута для любого $x \in \mathcal{I}_i$. Дополнительно предположим, что f' и f'' поддерживают вызовы EV. Тогда за $O(n)$ вызовов EV можно определить все точки перегиба. Далее разобьем последовательность b на участки выпуклости/вогнутости f_i. На каждом участке f_i строим разреженную таблицу. Далее при помощи f_i $\forall i \in [1, k + 1]$ вычисляем $c_j = min\{T_i.query(l_i, r_i, l_i - l_1)\}$, где l_i, r_i левые и правые границы запроса на i-той таблице, где для таблиц, находящихся в середине они совпадают с границами таблицы.

Используя утверждение Теоремы 3 получим итоговую сложность

$$O(k \cdot n^{5/3} \log_2(n)).$$

Теорема доказана. □

3.1.4 (min,+)-свертка для полиномиальной функции

Теорема 5 Пусть $f(x) \in \mathbb{Z}^d$ - полиномиальная функция. Будем полагать, что f поддерживает вызовы EV. Тогда (min,+)-свертка последовательностей $a := A$ и $b := b_i = f(i)$ может быть вычислена с арифметической и оракульной сложностью

$$O(d \cdot n^{5/3} \cdot \log_2(n)).$$

Доказательство

Так как $f \in \mathbb{Z}^d$, то f имеет d точек перегиба. Так как f - полиномиальная, то f бесконечное число раз дифференцируема в своей области определения. Таким образом, выполнены все условия Теоремы 3. Следовательно по Теореме 4 получаем, что итоговая сложность составляет

$$O(d \cdot n^{5/3} \cdot \log_2(n)).$$

операции.
Теорема доказана. □

3.2 Задача о (min,+)-свертке с ипсользованием дерева отрезков с модификацией

Далее приведем описание техники вычисления (min,+)-свертки при помощи дерева отрезков, представим и докажем полученые результаты.
Рассмотрим задачу о (min,+)-свертке для следующих случаев:

1. f - выпуклая/вогнутая;
2. f имеет k точек перегиба;
3. $f \in \mathbb{Z}^d[x]$.

3.2.1 (min,+)-свертка для выпуклых и вогнутых функций

Теорема 6 Пусть $f : [0, m) \to \mathbb{R}$ - выпуклая/вогнутая функция. Будем полагать, что f поддерживает вызовы EV.
Тогда (min,+)-свертка может быть вычислена с арифметической и оракульной сложностью

$$O(n^{4/3} \cdot \log_2^2(n)).$$

Доказательство

Доказательство теоремы можно найти в [1] в разделе 3.3 Теорема 3. □

3.2.2 (min,+)-свертка для функции с k точками перегиба

Теорема 7 Пусть $f : [0, n) \to \mathbb{R}$ функция, имеющая k точек перегиба. Будем полагать, что f поддерживает вызовы EV и имеет непрерывные производные до 2 порядка включительно.
Тогда (min,+)-свертка последовательностей $a := A$ и $b := b_i = f(i)$ может быть вычислена с арифметической и оракульной сложностью

$$O(k \cdot n^{4/3} \cdot \log_2^2(n)).$$

Доказательство

Так как функция f имеет k точек перегиба следовательно на $[0, n)$ имеем $k + 1$ участок $\mathcal{I}_i : f(x)$ выпукла или вогнута для любого $x \in \mathcal{I}_i$. Дополнительно предположим, что f' и f'' поддерживают вызовы EV. Тогда за $O(n)$ вызовов EV можно определить все точки перегиба. Далее разобьем последовательность b на участки выпуклости/вогнутости f_i. На каждом участке f_i строим дерево отрезков с модификацией. Далее при помощи f_i $\forall i \in [1, k+1]$ вычисляем $c_k = min\{T_i.query(l_i, r_i, l_i - l_1)\}$, где l_i, r_i левые и правые границы запроса на i-том дереве отрезков T_i, где для всех деревьев, находящихся в середине они совпадают с границами дерева.

Используя утверждение Теоремы 7 получим итоговую сложность

$$O(k \cdot n^{4/3} \log_2^2(n)).$$

Теорема доказана. □

3.2.3 (min,+)-свертка для полиномиальной функции

Теорема 8 Пусть $f(x) \in \mathbb{Z}^d$ - полиномиальная функция. Будем полагать, что f поддерживает вызовы EV. Тогда (min,+)-свертка последовательностей $a := A$ и $b := b_i = f(i)$ может быть вычислена с арифметической и оракульной сложностью

$$O(d \cdot n^{4/3} \cdot \log_2^2(n)).$$

Доказательство

Так как $f \in \mathbb{Z}^d$, то f имеет d точек перегиба. Так как f - полиномиальная, то f бесконечное число раз дифференцируема в своей области определения. Таким образом, выполнены все условия Теоремы 7. Следовательно по Теореме 8 получаем, что итоговая сложность составляет

$$O(d \cdot n^{4/3} \cdot \log_2^2(n)).$$

операции.
Теорема доказана. $\square$

3.3 Задача о рюкзаке

3.3.1 Техника динамического программироавния для задачи о рюкзаке

Рассмотрим следующую постановку задачи о рюкзаке. Пусть задана сепарабельная функция $f(x) : \mathbb{Z}^n \to \mathbb{R}$, т.е. $f(x) = f_1(x_1)+...+f_n(x_n)$, где $f_i(x_i) : \mathbb{Z} \to \mathbb{R}$. Также заданы векторы $w \in \mathbb{Z}^n_{>0}$, $u \in \mathbb{Z}^n_{>0}$ и целое число $W \in \mathbb{Z}$. Рассмотрим следующую задачу:

$$f(x) \to max$$

$$\begin{cases} \omega^\top x = W \\ 0 \leqslant x \leqslant u \\ x \in \mathbb{Z}^n. \end{cases}$$

Также, без ограничения общности рассмотрения будем полагать, что $u_k \leqslant \lfloor W/w_k \rfloor$ для $k \in \{1, ..., n\}$. Сведем пповставленную задачу к эквивалентной задаче минимизации, заменив исходный функционал f на $-f$. Для решения задачи воспользуемся принципом динамического программирования, аналогичного описанному в [1] и [4]. Для $k \in \{1, ..., n\}$ и $w_0 \in \{0, ..., W\}$ под $DP(k, w_0)$ определим следующую задачу:

$$\sum_{i=1}^{k} f_i(x_i) \to min$$

$$\begin{cases} \sum\limits_{i=1}^{k} w_i x_i = w_0 \\ 0 \leqslant x_i \leqslant u_i \\ x \in \mathbb{Z}^k. \end{cases}$$

Для $2 \leqslant k \leqslant n$ значение $DP(k, w_0)$ может быть вычислено при помощи ранее подсчитанных значений $DP(k-1, \cdot)$ по следующей формуле:

$$DP(k, w_0) = \min_{j \in \{0,...,u_k\}} \{DP(k-1, w_0 - w_k \cdot j) + f_k(j)\},$$

в предположении, что $DP(k, w_0) = +\infty$ для $w_0 < 0$. Вычислим ответы для состояний $k = 1$. Имеем

$$DP(1, w_0) = \begin{cases} f_1(w_0/w_1), & \text{если } w_1 | w_0 \text{ и } 0 \leqslant w_0/w_1 \leqslant u_1 \\ +\infty, & \text{иначе.} \end{cases}$$

Оценим сложность, необходимую для вычисления значений $DP(k, w_0)$ для всех $k \in \{1, ..., n\}$ и $w_0 \in \{0, ..., W\}$.
Значения $DP(1, w_0)$ могут быть вычислены за $O(W)$ арифметические операции. Каждое из значений $DP(1, 0), DP(1, w_1), DP(1, 2w_1)$ могут быть вычислены за $O(1)$. Для остальных значений $DP(1, w_0)$ положим $+\infty$.
Далее сведем вычисление остальных состояний к вычислению (min,+)-свертки. Описание сведения возьмем из [1].
Пусть $k \geqslant 2$. Зафиксируем некоторый остаток r по модулю w_k. Определим последовательноости $\{a_i\}_{i \in \{0,...,u_k\}}$, $\{b_i\}_{i \in \{u_k\}}$ и $\{c_i\}_{i \in \{0,...,u_k\}}$ следующим образом:

$$a_i = DP(k-1, r + i \cdot w_k),$$

$$b_i = f_k(i),$$

$$c_i = DP(k, r + i \cdot w_k).$$

Полагая, что $a_i = b_i = c_i$ для всех $i < 0$ и используя формулу для $DP(k, w_0)$ получаем

$$c_i = \min_{j \in \{0,\dots,i\}} \{a_{i-j} + b_j\}.$$

Таким образом, задача была сведена к вычислению $c = (a \star b)[0, u_k]$. Рассматривая всевозможные значения r сложность вычисления для вычисления всех состояний $DP(k, \cdot)$ при условии, что все состояния $DP(k-1, \cdot)$ были вычислены может быть вычислено за

$$O(w_k \cdot T_c(u_k)).$$

где T_c - сложность вычисления (min,+)-свертки. Итоговая сложность, для реализации техники динамического программирования составляет

$$O(W + \sum_{k=2}^{n} w_k \cdot T_c(u_k)).$$

Учитывая оценку $u_k \leqslant \lfloor W/w_k \rfloor$ можем получить следующее ограничение

$$O(W + \sum_{k=2}^{n} w_k \cdot T_c(W/w_k)) \tag{6}$$

3.3.2 Результаты для задачи о рюкзаке для функционала с ограничениями

Теорема 9 Пусть в задаче о рюкзаке функционал $f(x)$:

1. выпуклая или вогнутая функция (I)
2. $f \in \mathbb{Z}^d$ (II)

Тогда (I) может быть вычислена за $O(n \cdot W^{4/3} \cdot \log_2^2(W))$ при помощи дерева отрезков с модификацией и за $O(n \cdot W^{5/3} \cdot \log_2(W))$ при помощи разреженной таблицы;
(II) может быть вычислена за $O(d \cdot n \cdot W^{4/3} \cdot \log_2^2(W))$ при помощи дерева отрезков с модификацией и за $O(d \cdot n \cdot W^{5/3} \cdot \log_2(W))$ при помощи разреженной таблицы.

Доказательство

Доказательство следует из Теорем 3, 5, 6, 8 и (6). □

3.3.3 Задача о рюкзаке для функционала в общем случае

Далее рассмотрим еще один вариант постановки задачи о рюкзаке.
Пусть задан вектор $a = (a^{(1)}, ..., a^{(n)})^T \in \mathbb{Z}_+^n$
Рассмотрим следующее множество:

$$B_a^u(b) = \left\{ x \in \prod_{i=1}^{n} \{0, ..., u^{(i)}\} : \langle a, x \rangle = b \right\}, \tag{7}$$

где $\langle x, y \rangle = \sum\limits_i x^{(i)} \cdot y^{(i)}$ -скалярное произведение в векторном пространстве.
Введем следующую функцию:

$$f_{B_a^u}(t) = \sum_{b=0}^{\infty} \mathcal{N}(B_a^u(b)) \cdot t^b, t \in \mathbb{R}, \tag{8}$$

где $\mathcal{N}(Q)$ -число целочисленных точек на множестве $Q \subseteq \mathbb{R}^n$. Функцию f будем называть порождающей.

Лемма 2

$$f_{B_a^u}(t) = \prod_{i=1}^{n} \left(\sum_{k=0}^{u^{(i)}} t^{ka^{(i)}} \right)$$

Доказательство

Доказательство можно найти в [3] Лемма 1. □

Задача сводится к вычислению произведения многочленов. Для этого в [3] применяется алгоритм быстрого преобразования Фурье (FFT).

Теорема 10 Все $\langle a, u \rangle$ коэффициентов полинома $f_{B_a^u}(t)$ могут быть вычислены при помощи быстрого преобразования Фурье за

$$O(\langle a, u \rangle \ln \langle a, u \rangle \ln n) \tag{9}$$

арифметические операции.

Доказательство

Доказательство можно найти в [3] Теорема 1. □

Определение 10 Рассмотрим вектор $c \in \mathbb{R}^n$. Для конечного множества $\mathcal{R} \subset \mathbb{R}^n$ определим характеристическую функцию

$$g_{\mathcal{R}}(c) = \sum_{x \in \mathcal{R}} e^{\langle c, x \rangle} \tag{10}$$

и порождающую функцию

$$\psi_{\mathcal{R}}(c) = \ln g_{\mathcal{R}}(c).$$

Исходная задача сводится к поиску функции

$$f^* = \max_{x \in \mathbb{Z}_+^n} \{\langle c, x\rangle : \langle a, x\rangle = b\} \tag{11}$$

Определение 11 Дополненной порождающей функцией для векторов a и c будем называть функцию

$$F_{B_a^u}(c,t) = \sum_{b=0}^{\infty} g_{B_a^u(b)}(c) \cdot t^b = \sum_{b=0}^{\infty} exp(\psi_{B_a^u(b)}(c)) \cdot t^b \tag{12}$$

В [3] доказывается, что (11) имеет следующее представление:

$$F_{B_a^u}(c,t) = \prod_{i=1}^{n} \left(\sum_{k=0}^{u^{(i)}} e^{kc^{(i)}} \cdot t^{ka^{(i)}} \right)$$

В [3] описывается алгоритм, вычисляющий решение задачи (11) за

$$O(\|a\| \cdot \ln \|a\| \cdot \ln n + b \cdot \ln^2 n). \tag{13}$$

3.3.4 Метод двойного экспоненциального представления

Определение 12 Пусть $z \in \mathbb{C}$. Двойным экспоненциальным представлением числа z будем называть число $a \in \mathbb{C}$ или пару чисел $a, b \in \mathbb{C}$, при условии, что $||\arg(a)| - |\arg(b)|| \geqslant 2\pi/3$:

$$z = e^a \text{ или } z = e^a + e^b.$$

Двойное экспоненциальное представление числа $z = e^a$ будем называть *каноническим*.
Определение 13 Пусть $M \subseteq \mathbb{C}$ и функция $f : M \to \mathbb{C}$. Будем говорить, что функция f полиномиально аппроксимирующая, если существует алгоритм со следующими свойствами:

1. для любого $y \in M$ и для $\varepsilon \in \mathbb{Q}_{>0}$ алгоритм вычисляет значение z: $|z - f(y)| \leqslant \varepsilon$
2. сложность алгоритма ограничена полиномиальной функцией аргумента $size(y) + size(\varepsilon)$

Лемма 3 Пусть $M = \{(x,y) \in \mathbb{C}^2 : -2\pi/3 \leqslant |\arg(x) - \arg(y)| \leqslant 2\pi/3\}$. Пусть задана функция $f : M \to \mathbb{C}$, такая что для пары $(x,y) \in M$ функция $f(x,y)$ равняется единственному значению z, причем

$$e^z = e^x + e^y.$$

Тогда говорим, что функция полиномиально аппроксимирующая.

Доказательство

Доказательство леммы можно найти в [5]. □

3.3.5 Задача о рюкзаке с методом двойного экспоненциального представления

В статье [3] алгоритм имеет сложность (12) в предположении, что все операции могут быть выполнены за $O(1)$. Однако решение будет зависеть также от максимального значения c_{max} вектора c. Таким образом, оценка сложности алгоритма составляет

$$O((\|a\| \cdot \ln \|a\| \cdot \ln n + b \cdot \ln^2 n) \cdot c_{max}).$$

Для того, чтобы избавиться от экспоненциальной сложности предлагается построить вектор $c^* : c^{*(i)} = \lg c^{(i)}$ и решить задачу $f^* = \max\limits_{x \in \mathbb{Z}_+^n} \{\langle c^*, x\rangle : \langle a, x\rangle = b\}$ и по ней восстановить ответ для исходной задачи. При вычислении свертки в пространстве Фурье будем использовать метод двойного экспоненциального представления: если можно сложить элементы без потери точности, то будем их складывать, иначе храним в виде пары элементов. Для покрытия всех коэффициентов полинома $f*$ будем запускать рандомизированный алгоритм со случайным выбором первообразного корня матрицы Вандермонда. Задача получить все коэффициенты f в каноническом двойном экспоненциальном представлении. Для проверки гипотезы был реализован рандомизированный алгоритм. Используем алгоритм Кули-Тьюки при вычислении FFT.

```
#include <iostream>
#include <vector>
#include <istream>
#include <ostream>
#include <complex>
#include <algorithm>
#include <cmath>
#include <random>
#include <set>
#include <numeric>
using namespace std;
using ll = long long;
using ld = long double;
using cld = complex<long double>;
using vcld = vector<complex<long double>>;
long double pi = 3.1415926535898;
ld tol = 1e-3;
cld zero = { -16 };
ll randmax = 8;
random_device rd;
mt19937 gen(rd());
uniform_int_distribution<> dist(1, 200);

ld ang_comp(cld x, cld y)
{
    ld exr = abs(exp(x.real())), exi = abs(exp(x.imag()));
    if ((abs(exp(x.real())) <= tol && abs(exp(x.imag())) <= tol)
        || (abs(exp(y.real())) <= tol && abs(exp(y.imag())) <= 0))
        return (ld)0.;
    if (x.real() * y.real() >= tol && x.imag() * y.imag() >= tol)
        return abs(abs(arg(x)) - abs(arg(y)));
```

```
    if (x.real() * y.real() <= -tol && x.imag() * y.imag() <= -tol)
        return pi - ang_comp(x, -y);
    if (x.real() * y.real() <= -tol && x.imag() * y.imag() >= tol)
        return abs(arg(x) - arg(y));
    if (x.real() * y.real() >= tol && x.imag() * y.imag() <= -tol)
    {
        if (x.real() >= tol and y.real() >= tol)
            return abs(arg(x)) + abs(arg(y));
        if (x.real() <= -tol && y.real() <= -tol)
            return 2 * pi - abs(arg(x)) - abs(arg(y));
    }
}
ld anc(cld x, cld y)
{
    ld arg1 = atan2(x.imag(), x.real());
    ld arg2 = atan2(y.imag(), y.real());
    ld angle = arg2 - arg1;
    angle = fmod(angle + 2 * pi, 2 * pi);
    if (angle > pi)
        angle -= 2 * pi;
    return angle;
}

class DER
{
    cld a;
    cld b;
public:
    DER();
    DER(cld f, cld l);
    void print();
    void set_rand();
    DER& operator +=(DER& right);
    DER& operator *= (DER& right);
    DER operator=(DER right);
    DER operator+(DER right);
    DER operator*(DER& right);
    DER& operator/=(ld right);
    DER operator-(DER right);
    cld geta();
    cld getb();
    friend ostream& operator<<(ostream& stream, const DER& der);
};

using vd = vector<DER>;

DER::DER()
{
    this->a = zero;
    this->b = zero;
}
```

```
DER::DER(cld f, cld l = zero)
{
    a = f;
    b = l;
}

void DER::print()
{
    cout << this->a;
    if (abs(exp(this->b) - cld(0)) >= 1e-3)
        cout << " " << this->b;
    cout << endl;
}

void DER::set_rand()
{
    this->a = cld(dist(gen));
    this->b = zero;
}

DER& DER::operator+=(DER& right)
{
    vcld list;
    if (abs(exp(this->a)) >= 1e-3)
        list.push_back(this->a);
    if (abs(exp(this->b)) >= 1e-3)
        list.push_back(this->b);
    if (abs(exp(right.a)) >= 1e-3)
        list.push_back(right.a);
    if (abs(exp(right.b)) >= 1e-3)
        list.push_back(right.b);
    DER d;
    for (ll i = 0; i < list.size(); i++)
        list[i] = exp(list[i]);
    if (!(list.size()))
        return d;
    ll su = 0;
    while (1)
    {
        if (su == 100)
            ll foo = 0;
        su++;
        if (list.size() == 1)
            break;
        if (list.size() == 2)
        {
            if (anc(log(list[0]), log(list[1])) < 2 * pi / 3 + 0.1)
            {
                list[0] += list[1];
                list.pop_back();
            }
```

```
            break;
        }
        sort(list.begin(), list.end(), [](cld f1, cld f2)
            { return arg(log(f1)) < arg(log(f2)); });
        for (ll i = 0; i < list.size(); i++)
        {
            if (anc(log(list[i]), log(list[(i + 1) % list.size()]))
                < 2 * pi / 3 + 0.1)
            {
                list[i] += list[(i + 1) % list.size()];
                list.erase(list.begin() + (i + 1) % list.size());
            }
        }
    }
    this->a = log(list[0]);
    this->b = zero;
    if (list.size() == 2)
        this->b = log(list[1]);
    return *this;
}

DER DER::operator=(DER right)
{
    this->a = right.a;
    this->b = right.b;
    return *this;
}

DER& DER::operator *=(DER& right)
{
    vcld list(4), e_list(4);
    list[0] = this->a + right.a;
    list[1] = this->a + right.b;
    list[2] = this->b + right.a;
    list[3] = this->b + right.b;
    DER ans1, ans2;
    ans1.a = list[0];
    ans1.b = list[1];
    ans2.a = list[2];
    ans2.b = list[3];
    ans1 += ans2;
    *this = ans1;
    return *this;
}

DER DER::operator+(DER right)
{
    DER ans = *this;
    ans += right;
    return ans;
}
```

```
DER DER::operator*(DER& right)
{
    DER ans = *this;
    ans *= right;
    return ans;
}

DER& DER::operator/=(ld right)
{
    cld f = exp(this->a), s = exp(this->b);
    f /= right, s /= right;
    this->a = log(f), this->b = log(s);
    return *this;
}

DER DER::operator-(DER right)
{
    DER t2(log(-exp(right.a)), log(-exp(right.b)));
    return *this + t2;
}

ostream& operator<<(ostream& stream, const DER& der)
{
    stream << der.a;
    if (abs(exp(der.b) - cld(0)) >= 1e-3)
    {
        stream << " + " << der.b;
    }
    return stream;
}

cld DER::geta()
{
    return this->a;
}

cld DER::getb()
{
    return this->b;
}

DER root(ll rank, ll pow)
{
    cld j1 = { 0, 1 };
    DER res(exp((2. * pi * pow / rank) * j1));
    return res;
}

vd fft(vd x, ll u, bool invert)
{
```

```
    ll n = x.size();
    if (x.size() == 1)
        return x;
    vd ev(x.size() / 2), od(x.size() / 2);
    for (ll i = 0, j = 0; i < x.size(); i += 2, j++)
    {
        ev[j] = x[i];
        od[j] = x[i + 1];
    }
    ev = fft(ev, u, invert);
    od = fft(od, u, invert);
    ld ang = 2. * u * pi / n * (invert ? -1 : 1);
    DER w(0), wn(log(cld{ cos(ang), sin(ang) }));
    for (ll i = 0; i < n / 2; i++)
    {
        x[i] = ev[i] + od[i] * w;
        DER temp = (w * od[i]);
        x[n / 2 + i] = (ev[i] - temp);
        if (invert)
        {
            x[i] /= 2;
            x[n / 2 + i] /= 2;
        }
        w *= wn;
    }
    return x;
}

vd mult(vd x, vd y, ll u)
{
    ll n = 1;
    while (n < max(x.size(), y.size())) n <<= 1;
    n <<= 1;
    x.resize(n), y.resize(n);
    reverse(x.begin(), x.begin() + n / 2);
    reverse(y.begin(), y.begin() + n / 2);
    reverse(x.begin(), x.end());
    reverse(y.begin(), y.end());
    x = fft(x, u, 0), y = fft(y, u, 0);
    for (ll i = 0; i < n; i++)
        x[i] *= y[i];
    x = fft(x, u, 1);
    return x;
}

vector<cld> multsimple(vd x, vd y)
{
    for (ll i = 0; i < x.size(); i++)
        if ((abs(exp(x[i].getb()) - cld(0)) >= 1e-3))
        {
            DER tmp(log(exp(x[i].geta()) + exp(x[i].getb())));
```

```
            x[i] = tmp;
        }
    for (ll i = 0; i < x.size(); i++)
        if ((abs(exp(y[i].getb()) - cld(0)) >= 1e-3))
        {
            DER tmp(log(exp(y[i].geta()) + exp(y[i].getb())));
            y[i] = tmp;
        }
    vector<cld> le(x.size()), ri(y.size());
    for (ll i = 0; i < x.size(); i++)
        le[i] = exp(x[i].geta());
    for (ll i = 0; i < y.size(); i++)
        ri[i] = exp(y[i].geta());
    vector<cld> ans(x.size() + y.size());
    for (ll i = 0; i < le.size(); i++)
        for (ll j = 0; j < ri.size(); j++)
            ans[i + j] += le[i] * ri[j];
    return ans;
}

int main()
{
    DER f(cld{ 1, 1 });
    vd a(2), b(2);
    a[0] = DER(1), a[1] = DER(1);
    b[0] = DER(1), b[1] = DER(1);
    a = mult(a, b, 1);
    for (ll i = 0; i < a.size(); i++)
        cout << a[i] << endl;
    ll n;
    cin >> n;
    vector<ll> proot;
    for (ll i = 1; i < n; i += 2)
        proot.push_back(i);
    vector<ll> res1;
    vector<ll> resh;
    vector<ll> res7;
    for (ll i = 0; i < 200; i++)
    {
        vd a(n / 2), b(n / 2);
        vector<ll> ap(n / 2), bp(n / 2);
        for (ll i = 0; i < n / 2; i++)
        {
            ap[i] = dist(gen);
            bp[i] = dist(gen);
        }
        for (ll i = 0; i < n / 2; i++)
        {
            DER tmp(log(ap[i]));
            a[i] = tmp;
            DER tmp1(log(bp[i]));
```

```
        b[i] = tmp1;
    }
    vector<bool> check(n);
    ll halfcovstep = 50;
    ll sevcovstep = 50;
    ll fcovstep = 0;
    bool chalf = 0;
    bool c7 = 0;
    for (ll i = 0; i < 50; i++)
    {
        uniform_int_distribution<> distrib(0, proot.size() - 1);
        ll j = proot[distrib(gen)];
        vd c = a, d = b;
        c = mult(c, d, j);
        for (ll i = 0; i < c.size(); i++)
            if ((abs(exp(c[i].getb()) - cld(0)) < 1e-2))
                check[i] = 1;
        if (i == 0)
        {
            cout << "Closed 1st step: " <<
                accumulate(check.begin(), check.end(), 0) << endl;
            fcovstep = accumulate(check.begin(), check.end(), 0);
        }
        if (accumulate(check.begin(), check.end(), 0) >= n / 2 && !chalf)
        {
            cout << "1 half covered steps: " << i << endl;
            halfcovstep = i + 1;
            chalf = 1;
        }
        if (accumulate(check.begin(), check.end(), 0) >= 0.99 * n && !c7)
        {
            cout << "99 % covered steps " << i << endl;
            sevcovstep = i + 1;
            c7 = 1;
            break;
        }

    }
    res1.push_back(fcovstep);
    resh.push_back(halfcovstep);
    res7.push_back(sevcovstep);
}
cout << endl;
cout << "1 step\n";
for (ll i = 0; i < res1.size(); i++)
    cout << res1[i] << ' ';
cout << endl;
cout << "1/2 step\n";
for (ll i = 0; i < resh.size(); i++)
    cout << resh[i] << ' ';
cout << endl;
```

```
    cout << "99 % step\n";
    for (ll i = 0; i < res7.size(); i++)
        cout << res7[i] << ' ';
    cout << "\n\n\nE on 1st step: "
        << 1. * accumulate(res1.begin(), res1.end(), 0) / res1.size() << "\n\n\n";
    cout << "\n\n\nE on 1/2: "
        << 1. * accumulate(resh.begin(), resh.end(), 0) / resh.size() << "\n\n\n";
    cout << "\n\n\nE on 99 %: "
        << 1. * accumulate(res7.begin(), res7.end(), 0) / res7.size() << "\n\n\n";
    cout << "\n\n\nUNCOVERED: " << count(res7.begin(), res7.end(), 50) << "\n\n\n";
}
```

Для проверки корректности полученных результатов был реализован алгоритм умножения многочленов столбиком:

```
vector<cld> multsimple(vd x, vd y)
{
    for (ll i = 0; i < x.size(); i++)
        if ((abs(exp(x[i].getb()) - cld(0)) >= 1e-3))
        {
            DER tmp(log(exp(x[i].geta()) + exp(x[i].getb())));
            x[i] = tmp;
        }
    for (ll i = 0; i < x.size(); i++)
        if ((abs(exp(y[i].getb()) - cld(0)) >= 1e-3))
        {
            DER tmp(log(exp(y[i].geta()) + exp(y[i].getb())));
            y[i] = tmp;
        }
    vector<cld> le(x.size()), ri(y.size());
    for (ll i = 0; i < x.size(); i++)
        le[i] = exp(x[i].geta());
    for (ll i = 0; i < y.size(); i++)
        ri[i] = exp(y[i].geta());
    vector<cld> ans(x.size() + y.size());
    for (ll i = 0; i < le.size(); i++)
        for (ll j = 0; j < ri.size(); j++)
            ans[i + j] += le[i] * ri[j];
    return ans;
}
```

3.3.6 Результаты эксперимента

Проверка показала, что после завершения алгоритма все коэффициенты хранятся в сумме не более, чем двух слагаемых и разница между коэффициентами полученными при применении FFT и умножении столбиком не превосходит по модулю 10^{-3}.
Исследование проводилось для полиномов длины $n = 2^k$, где $k \in [5, 13]$. В ходе эксперимента для каждого значения выборка генерировалась 200 раз. Выбор первообразного корня проводился 50 раз. Была получена вероятность покрыть 99% коэффициентов случайным выбором первообразного корня; среднее число шагов, необходимых для покрытия 1/2 коэффициентов; среднее число шагов, необходимых для покрытия 99% коэффициентов; число коэффициентов, покрытых на первом шаге; исследована зависимость

полученных результатов от длины полинома. Результаты представим в табличной форме:

n	Вероятность покрыть все к-ты	E(1 шаг)	E(1/2)	E(0.99)
32	47.5	21.95	1.01	7.10465
64	72.5	39.9	1.045	9.43537
128	97.5	75.575	1.05	11.2083
256	99.5	140.48	1.115	12.77
512	100	282.795	1.075	11.075
1024	100	587.505	1	11.075
2048	100	1172.18	1	13.27
4096	100	2348.1	1	12.69
8192	100	4720.05	1	11.9

Представим полученные результаты в графической форме.

Графическое представление результатов:

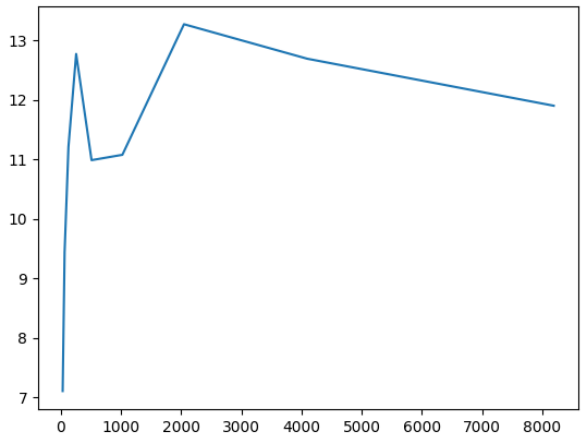

Рис. 1: Зависимость числа шагов для покрытия 99 % коэффициентов от длины полинома

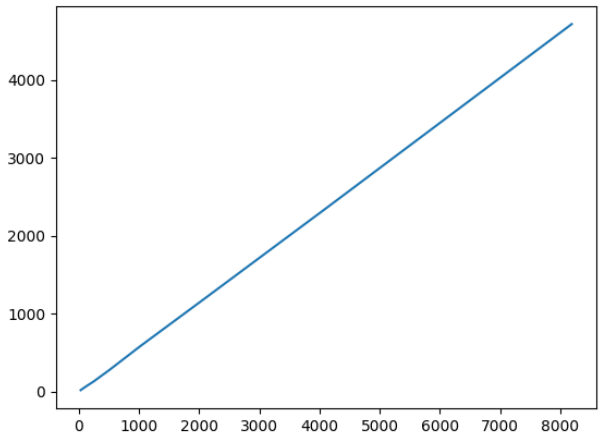

Рис. 2: Зависимость доли покрытия на первом шаге от длины полинома

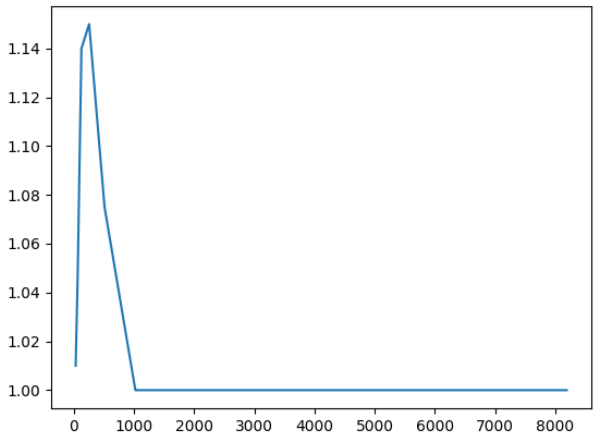

Рис. 3: Зависимость числа шагов для покрытия 1/2 коэффициентов от длины полинома

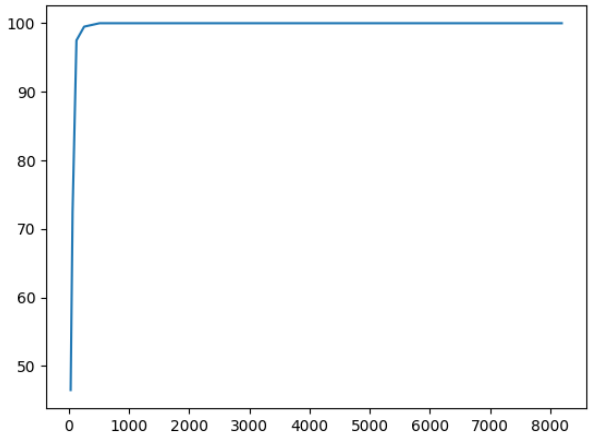

Рис. 4: Зависимость вероятности покрытия всех коэффициентов от длины полинома

По полученным результатам можно выдвинуть предположение о линейной зависимости количества покрытых коэффициентов полинома от длины полинома. Начиная с длины полинома 512 вероятность покрыть все коэффициенты достигает 100%. Необходимое число запусков алгоритма для покрытия 99% коэффициентов ограничено константой. 1/2 всех коэффициентов может быть покрыта не более чем за 2 шага.

4 Выводы

В результате работы была получен субквадратичный алгоритм, использующий технику вычисления (min,+)-свертки при помощи разреженной таблицы. Была улучшена оценка сложности субквадратичного алгоритма вычисления (min,+)-свертки при помощи модифицированного дерева отрезков для полиномиальной последовательности. Были получены субкубические алгоритмы для задачи о рюкзаке. Был проведен вычислительный эксперимент модификации существующего алгоритма.

5 Ссылки и использованная литература

1. Gribanov D. V., Shumilov I.A., Malyshev D.S.:Structured (min,+)-Convolution And Its Applications For The Shortest Vector, Closest Vector, and Separable Nonlinear Knapsack Problems (2022)

2. Bremner, D., Chan, T.M., Demaine, E.D., Erickson, J., Hurtado, F., Iacono, J., Langer man, S., Taslakian, P.: Necklaces, convolutions, and x + y. In: European Symposium on Algorithms, pp. 160–171. Springer (2006)

3. Nesterov, Y.: Fast fourier transform and its applications to integer knapsack problems (2004)

4. Bellman, R.: Dynamic programming. Science 153(3731), 34–37 (1966)

5. `https://www.overleaf.com/project/64f1c8504e308d360408c191`

6. E. B. Saff, A. Shönhage, R. S. Varga - Geometric Convergence to e^{-z} by Rational Functions with Real Poles - 18.07.1975

7. `https://algorithmica.org/ru/sparse-table`

8. `http://e-maxx.ru/algo/fft_multiply`